



Homework 8
—
**Artificial Neural Networks
(ANN)**
—
SOLUTIONS



Artificial Neurons for Boolean Function

♦ Solutions

- 1) What does an artificial neuron look like to implement the Boolean OR function?

This neuron is similar to the neuron for AND: it has the same inputs, only the weights are different: $w_0 = \text{any value between } 0 \text{ and } 1, w_1 = 1, w_2 = 1$.

- 2) What does an artificial neuron look like to implement the Boolean NOT function?

This neuron only has one real input x_1 (which can be 0 or 1) in addition to the bias $x_0 = -1$. w_0 then has to be smaller than 0. w_1 can in turn be any number smaller than w_0 . Once again, there are infinitely many weights implementing this function.



Structure of ANN

Solutions

- 1) How many input neurons and how many output neurons does the example used in the video have? Why exactly this many?
Input Neurons: $28 \times 28 = 784$, because the input image has that many pixels.
Output Neurons: 10, because we have 10 different classes of images (one class for each possible number)
- 2) What range of values can the activation of each neuron take on?
[0.0, 1.0]
- 3) Does it matter, in which order the pixels of the input image are fed into the network? Why/Why not?
No, it does not matter at all, BUT the pixels of all images have to be fed into the network in the same order.
- 4) He chose 2 hidden layers with 16 neurons each, admitting that this is a quite arbitrary choice. What do you expect to happen, if the number of hidden layers and the number of neurons in them is increased?
This is a harder question. With increasing number of hidden layers and neurons in them, the "capacity" of the network to remember increases. This means, that the quality may improve, if the capacity was too small for the problem the network is supposed to solve. However, if we keep increasing, the network may start to memorize the examples it has seen, and become worse for images, it has not seen. The neural network classifier starts to overfit.
- 5) Why does your network with 784 input neurons and 16 neurons in the second layer have 12560 weights between these layers, and not $12544 = 784 \times 16$?
The additional 16 "weights" are the biases. Remember the "trick" shown in Homework Notebook 1 we used to not have to deal with them in a special way!
- 6) What is the motivation given for having exactly 2 hidden layers?
The motivation is that the first hidden layer should pick up on edges in the input pixels and the second hidden layer picks up on patterns composed of these edges.
- 7) In the video when moving to the matrix/vector multiplication, he is keeping the biases as a separate vector (added on). In the first Homework Notebook, we saw the trick to treat the bias just like all other weight. How do we need to change the matrix W and the activation vector a to eliminate the bias vector b from the formula?
We need to add another column to the matrix W , e.g. as a first column, with the values from the vector b
In addition, we need to add a constant activation of 1 (one) to the activation vector in the same place (e.g. as the first value).
- 8) In summary, what is a neural network?
A neural network is just a function taking (usually very many) input values and computing (usually only a few) output values. It is an absurdly complicated function, though, with thousands or even millions of parameters (weights and biases).



Gradient Descent (revisited)

♦ Solutions

- 1) How do we initialize the weights and biases of a network at the start of the training phase? How good do we expect this network to perform?
We initialize them randomly and expect a horrible performance (i.e., random results)
- 2) Why do we need a "cost function"?
In order to tell the computer how well it performed
- 3) How is the "cost of one training example" defined mathematically?
As the sum of the squares of the differences between the values computed by the network and the expected (target) values.
- 4) What does it mean if this cost is small, what if it is large?
If it is small, the network has given a relatively correct answer, if it is large, it has returned a mostly incorrect result.
- 5) How do we compute the cost of ALL the training examples? What is the meaning of this value?
By averaging the cost of all the individual training examples. This is a measure of how the network performs over ALL training examples (the higher, the worse).
- 6) What does the Gradient of a multidimensional function tell you? How does this help us in minimizing the cost function?
The Gradient tells you the direction of steepest ASCENT. By stepping into the direction of the NEGATIVE gradient, we can decrease the value of the cost function, thereby moving towards a LOCAL minimum, by doing this repeatedly.
- 7) What do we mean mathematically, if we talk about "a neural network is learning"?
We talk about the fact that it is simply minimizing a cost function.
- 8) What is "Gradient Descent"?
Gradient Descent is the process of repeatedly nudging the input of a function by some multiple of the negative gradient. It is a way to converge towards the local minimum of a cost function.
- 9) Why does a neural network give confident answers for the classification problem? How could you try to solve this problem?
Because its entire universe (the training data it has seen) consists of clearly defined handwritten digits only - it simply has never seen any random inputs, so it had no incentive to recognize these (mathematically, the cost function does not contain them).
We could try to solve this problem by including many random images in the training set and adding an 11th neuron ("not a number between 0 and 9" = "random image" = "I have no idea what this is") to the output layer. This way, the cost function will try to minimize this. But: we really need all kinds of different random images for this to work, not only "white noise static"!